Write-up: Model Documentation

1. Who are you?

- Name (first and last name or first name and last initial): Mitchell DeHaven
- Hometown: Brighton, CO

I'm Mitchell DeHaven, I work as a Machine Learning Engineer at a health startup currently. I primarily work on speech and NLP topics.

2. Motivation

It sounded like a very challenging domain for speech work., specifically low resource, out of domain, and potentially atypical speech.

3. High-Level Approach

My winning solution was fairly simple. It was a single model solution, Qwen 3 ASR 1.6B, with additional data generating via Qwen 3 TTS. The additional audio was generating via single word transcripts which was generated from Claude specifically attempting to generate an exhaustive list of common diagnostic words used by speech language pathologists.

4. Visualizations

I do not [have any visualizations].

5. Three Most Impactful Code Segments

```
None
        kl_alpha_max = self.hparams.kl_alpha
        kl_alpha_min = self.hparams.kl_alpha_min
        if kl_alpha_max > 0:
            import math
            progress = self.global_step /
max(self.trainer.estimated_stepping_batches, 1)
            kl_alpha = kl_alpha_min + (kl_alpha_max - kl_alpha_min) * 0.5 * (1 +
math.cos(math.pi * progress))
            with torch.no_grad():
                base_logits = self._get_base_logits(batch)

            if base_logits is not None:
                labels = batch["labels"]
```

```
                mask = labels != -100
                ft_log_probs = F.log_softmax(out.logits[mask], dim=-1)
                base_probs = F.softmax(base_logits[mask], dim=-1)
                kl_loss = F.kl_div(ft_log_probs, base_probs,
reduction="batchmean")
```

I used KL divergence as a regularization mechanism during training. The idea behind this is to minimize divergence from the base model, which is quite good as a baseline, but nudging the model toward the target task. I used a scheduler to anneal the weight of the loss to 0 near the end the training, as much of the divergence occurs in the beginning of model training when the gradients are large.

```
None
class DurationBucketSampler(Sampler):
```

Although it doesn't have a final impact of the model performance, it was probably top 3 since duration-based sampling reduced training time by ~33% for my setup compared to a simple random batch + padding setup. That reduction allowed me to run a lot more experiments.

```
None
N/a, probably the entire generate_tts.py script
```

The final but probably most impactful piece of the solution was the addition of synthetic data generated via TTS. The source words were generated by Claude attempting to build an exhaustive list of diagnostic words (commonly used by speech language pathologists) for identify speech pathologies. The reference speaker vectors were generated from audio samples in the dataset.

6. Machine Specs & Time

- CPU (model): AMD Ryzen Threadripper 7960X 24-Cores
- GPU (model or N/A): NVIDIA RTX PRO 6000 Blackwell Workstation Edition
- Memory (GB): 128
- OS: Ubuntu 24.04 Server
- Train duration: 17 hours
- Inference duration: ~2 hours for test set

7. Caveats & Known Issues

I ran into what appeared to be issues with Qwen ASR when trained on Blackwell (my GPU) and performing inference on Ampere (the A100s used for submission evaluation). I couldn't quite pin it down, but I was consistently getting 1-1.5 WER degradation when running the same Docker evaluation process on the smoketest locally and then the smoketest running on the server. Other than that, I didn't notice anything else.

8. Tools for Data Preparation / EDA

None.

9. Performance Evaluation Beyond Competition Metric

None.

10. Things Tried That Didn't Make Final

I originally thought I could expand the dataset (non-synthetically) by finding children audio recordings in the wild. The only large scale dataset that had a permissive license with the competition requirements was Emilia-Yodas which has ~114k hours of audio. I built a age classifier using Qwen TTS's speaker vector extractor and extracted ~ 450 hours of audio. However it didn't provide really any improvement, so I didn't use the audio in the final submission.

11. Future Improvements

The only major idea that comes to mind was to use an age-specific LoRa adapter per child age group. The test time metadata didn't provide age groups, but I had an age classifier which could have been used for determining the LoRa router. I think this would give quite the additional boost.

12. Simplifications for Faster Inference

Some form of distillation probably makes the most sense. Similar to distil-whisper, reducing the decoder depth via distillation would likely improve speed substantially without sacrificing too much accuracy. I spent some time on a batched audio tower as well which I think improved training / inference time by a decent margin, although on the blind test it degraded performance much more than I saw in local validation.